

CLAUDE CERTIFIED ARCHITECT

Foundations (CCA-F) — Exhaustive Study Guide

An unofficial, comprehensive exam-prep compendium covering all five blueprint domains: agent architecture & orchestration, tool design & MCP integration, Claude Code configuration & workflows, prompt engineering & structured output, and context management & reliability — plus a glossary, anti-pattern cheat sheets, a 12-week study plan, and 40 original practice questions with rationales.

Certification	Claude Certified Architect — Foundations (CCA-F / CCAR-F)
Issuer	Anthropic, via the Claude Partner Network
Launched	March 12, 2026
Format	Proctored, closed-book, no-AI-assistance exam — 60 scenario-based multiple-choice questions across 4 sections
Level	Level-300 (practitioner/production depth, not an intro literacy badge)
Passing score	720 / 1000
Access	Gated to verified Claude Partner Network member-company employees (free for the first wave; broader access to come)
Audience	Solution architects, senior AI/ML engineers, technical leads, and Claude Code power users designing production-grade AI systems

How to use this guide: Read domain-by-domain in weight order (largest first). Each domain section pairs core concepts with the anti-patterns the exam specifically probes — in this exam, the wrong answers are almost always a plausible-sounding shortcut, and the right answer almost always fixes the root cause rather than bolting on a workaround. Work the practice questions cold, then re-read the rationale for anything you miss.

This is an independent study aid, not an official Anthropic publication. Exam blueprint weights and question style are based on publicly reported candidate accounts current as of mid-2026; verify current exam details against your Partner Network portal before sitting the exam.

Table of Contents

Exam Overview & Blueprint	
Domain 1 — Agent Architecture & Orchestration	27%
Domain 2 — Claude Code Configuration & Workflows	20%
Domain 3 — Prompt Engineering & Structured Output	20%
Domain 4 — Tool Design & MCP Integration	18%
Domain 5 — Context Management & Reliability	15%
Cross-Domain Anti-Pattern Cheat Sheet	
Glossary of Terms	
12-Week Study Plan	
40 Practice Questions with Rationales	
Exam-Day Checklist & Final Tips	

Exam Overview & Blueprint

Anthropic introduced the Claude Certified Architect (CCA) program on March 12, 2026, alongside the \$100M Claude Partner Network investment. CCA Foundations is the first credential in what Anthropic describes as a broader stack of certifications for architects, developers, and sellers. It is deliberately not an "AI literacy" badge: it is a closed-book, no-AI-assistance, scenario-based exam aimed at people who design and operate production Claude systems — not people who can merely describe what an LLM is.

The exam is delivered as 60 multiple-choice questions split across four scenario-based sections, proctored, with a passing threshold of 720 out of 1000 points. Anthropic notes it is a Level-300 exam — assuming hands-on experience with the Claude API, Claude Code, and MCP, not introductory familiarity. Access currently runs through the Claude Partner Network: candidates register with a verified partner-company email, and the underlying training draws on Anthropic Academy material distributed through partners such as Fractal Analytics, Accenture, Deloitte, Cognizant, and Infosys.

Blueprint weighting

The five domains are not weighted evenly, and the weighting is itself informative: prompt engineering — the skill most people associate with "using Claude well" — sits in the middle of the pack. The heaviest domain is orchestration, because the exam assumes you can already write a reasonable prompt and instead tests whether you can build a multi-component system that keeps working when a subagent fails, a tool call times out, or a context window fills up.

Domain	Weight	Core question it tests
1. Agent architecture & orchestration	27%	Can you decompose, delegate, and recover in multi-agent systems?
2. Claude Code configuration & workflows	20%	Can you configure Claude Code for team-scale, CI-safe, context-efficient use?
3. Prompt engineering & structured output	20%	Can you get reliably correct, schema-valid output at scale?
4. Tool design & MCP integration	18%	Can you design tools and MCP servers an agent will call correctly?
5. Context management & reliability	15%	Can you keep a long-running agent accurate and know when to escalate?

The exam's real pattern

Across all five domains, the wrong answers repeat a shape: they are technically plausible fixes that add a new layer of infrastructure on top of a root cause instead of fixing the root cause. The graders reward the boring, structural fix over the clever one. Internalize this decision rule before you memorize anything else:

- If two tools are being confused, fix the **descriptions** before you build a routing classifier.
- If a policy must always hold, use **programmatic enforcement** (a hook, a schema, code) before you strengthen a system-prompt instruction.
- If a subagent lacks information, **pass it explicitly** in the task prompt before you build a shared memory layer.
- If an error propagates badly, fix what the **tool returns** before you add a supervisory retry wrapper.

Agent Architecture & Orchestration

This is the largest domain and the one candidates most often underestimate. It covers the mechanics of the agentic loop, when to reach for agents at all, how to decompose and delegate work, multi-agent topology, and where policy enforcement belongs.

1.1 What makes a system "agentic"

An agentic system is defined by three properties working together: the model has meaningful **autonomy** over what step to take next, it can invoke **tools** to change or observe the world beyond the conversation, and it operates inside a **loop** that keeps running until a task-defined stopping condition is met — not a fixed number of turns. A single-turn conversational exchange or a fixed linear pipeline (call model, call one API, format, return) is not agentic even if it uses the same underlying model. The exam distinguishes three tiers:

- **Conversational system** — a human drives every turn; the model never decides to act unprompted.
- **Workflow** — a fixed, human-authored sequence of steps (possibly using the LLM inside a step) with no runtime branching driven by the model's own judgment.
- **Agent** — the model itself decides which tools to call, in what order, and when it is done, adapting that plan as new information arrives.

When to use agentic architecture vs. a simpler pattern

The exam rewards restraint. Agentic architecture adds latency, cost, and failure surface, so it is justified when the task has open-ended scope, requires multiple dependent tool calls whose order can't be fixed in advance, or needs the system to adapt its plan based on intermediate results. A workflow (or even a single well-structured prompt) is the right call when the steps are fully knowable ahead of time — a fixed ETL pipeline with an LLM classification step in the middle does not need to be an agent.

1.2 The agentic loop mechanics

The Claude API surfaces the loop state through the **stop_reason** field on a response:

```
stop_reason: "tool_use" -> the model wants to call one or more tools.
Execute them, return tool_result blocks, and
send the conversation back to the model.
stop_reason: "end_turn" -> the model has produced its final answer.
Stop the loop.
stop_reason: "max_tokens" -> the response was truncated. Do not treat
this as completion.
stop_reason: "stop_sequence" -> a configured stop string was hit.
```

The exam explicitly tests two anti-patterns here, both of which look reasonable at first glance:

✗ Determining task completion by parsing the assistant's natural-language text for phrases like "I'm done" instead of checking **stop_reason** programmatically.

✗ Capping the loop at an arbitrary fixed iteration count and treating that as "completion" rather than a safety ceiling that should trigger explicit escalation when hit.

✓ Loop on **stop_reason == tool_use**, stop cleanly on **end_turn**, and treat a hit iteration ceiling as a distinct failure mode to surface, not a silent success.

1.3 Task decomposition & delegation

Good decomposition splits a task along boundaries that map to independently verifiable subtasks with minimal shared state. Bad decomposition splits along the first structure that comes to mind (e.g., “three examples”) without checking whether that structure actually covers the requested scope.

Worked failure scenario (high-yield for the exam)

A coordinator is asked to research “AI’s impact on creative industries.” It decomposes the task into three subagent assignments: AI in digital art, AI in graphic design, and AI in photography. Each subagent does excellent, accurate work. The final synthesized report, however, only covers visual art — it never touches music, writing, film, or performance. Where is the fault?

Root cause: the coordinator, not the subagents and not the synthesis step. The subagents did exactly what they were assigned. The coordinator silently narrowed a broad topic to a convenient subset at decomposition time and never assigned anything else. This is the single most representative failure pattern in the domain: garbage-in decomposition produces confidently correct, incompletely scoped output, and every downstream component looks like it worked.

Lesson for the exam: when a scenario asks you to find the root cause of an incomplete or skewed multi-agent result, check the **decomposition step first**, before blaming synthesis, prompting, or the subagents’ tool use.

1.4 Multi-agent topology

- **Hub-and-spoke (orchestrator–subagent)** — the default, exam-favored pattern. A coordinator owns task decomposition, delegation, error handling, and result aggregation. Subagents execute in **isolated context**: they inherit nothing automatically from the coordinator’s conversation. Anything a subagent needs — facts, constraints, prior findings, formatting requirements — must be explicitly packed into its task prompt, or it effectively does not exist for that subagent.
- **Pipeline** — sequential handoff, output of stage N becomes input to stage N+1. Good for well-ordered transformations (extract → validate → summarize); poor fit when steps need to run concurrently or revisit earlier stages.
- **Peer-to-peer** — agents communicate directly without a central coordinator. Highest flexibility, highest risk of duplicated work, race conditions, and unbounded message loops; requires the most deliberate guardrails and is rarely the exam’s “correct” answer unless the scenario explicitly calls for decentralized negotiation.

Context isolation is the load-bearing idea in this section. Because subagents don’t share memory with the coordinator or each other by default, coordinator prompts must explicitly restate relevant facts, and subagent context budgets must be sized for what that subagent alone needs — not the whole system’s context.

1.5 Sequential vs. parallel execution, planning & replanning

Run subagents in parallel when their subtasks are independent and don’t need each other’s intermediate output (e.g., three independent research streams). Run sequentially when a later step genuinely depends on an earlier step’s result (e.g., validate before you write). Parallelizing dependent steps is a common exam distractor — it looks like a performance win but silently drops the dependency.

Dynamic planning means the coordinator can revise its plan mid-execution when a subagent returns an unexpected result, a tool fails, or new information changes scope. A system that commits to a fixed plan up front and executes it blind to intermediate results handles ambiguity poorly; the exam favors coordinators that treat the plan as a living object, re-evaluated after each major step.

1.6 Escalation, ambiguity handling & policy enforcement

This subsection overlaps with Domain 5 but the exam also tests it from the orchestration angle: **where** a policy is enforced matters as much as what the policy says.

Worked scenario

A support agent must never process a refund over £500 without human approval. Two implementations are proposed: (a) a system-prompt instruction telling the model not to process refunds above the threshold, or (b) a **PreToolUse hook** that programmatically blocks the **process_refund** tool call whenever the amount exceeds £500, before it ever reaches execution.

A system-prompt instruction is probabilistic — it holds *most of the time*, and the failure mode is silent (the model just complies with an edge-case framing it wasn't trained to resist). A hook is deterministic — it fires on *every* call regardless of how the model was talked into requesting it. When the cost of a miss is financial, legal, or safety-critical, the exam wants the deterministic, programmatic control, not a stronger-worded prompt.

- Reliable, exam-favored controls: **hooks** (PreToolUse/PostToolUse), schema validation, code-level checks, allow/deny lists enforced outside the model.
- Unreliable controls the exam uses as distractors: relying solely on system-prompt wording, self-reported model confidence, or “please double-check yourself” instructions for hard policy limits.

1.7 Orchestrator-subagent responsibilities at a glance

Responsibility	Owner
Task decomposition & scoping	Coordinator
Delegation & context packing per subagent	Coordinator
Executing a narrow, well-scoped subtask	Subagent
Error handling / retry policy across subagents	Coordinator
Result aggregation & synthesis	Coordinator
Escalation to a human	Coordinator (via explicit, reliable triggers — see Domain 5)

Claude Code Configuration & Workflows

This domain tests whether you can configure Claude Code for team-scale, CI-safe, and context-efficient use — not whether you can write a single prompt. Expect scenario questions about misconfigured hierarchy, context bloat, and non-interactive automation.

2.1 The CLAUDE.md hierarchy

Claude Code resolves project context from three levels, each with a different scope and lifecycle:

Level	Path	Scope	Version-controlled?
User	~/.claude/CLAUDE.md	Personal preferences, applies across all your projects	No — local to you
Project	.claude/CLAUDE.md	Team conventions, shared across everyone working on the repo	No — not committed
Directory	CLAUDE.md in a subfolder	Conventions specific to that part of the codebase	Yes — committed

Worked scenario

A new developer joins a team and consistently fails to follow the project's established coding conventions, even though “the conventions are documented.” Most likely cause: whoever wrote them added the conventions to their own **user-level** CLAUDE.md rather than the **project-level** file — so the rules only ever applied to that one person's sessions and were never shared with the team. Fix: move team conventions into **.claude/CLAUDE.md** and commit it.

2.2 Conditional rules with .claude/rules/

Rules in **.claude/rules/** use YAML frontmatter to load conditionally based on file path glob patterns, keeping context lean by only injecting conventions relevant to the files actually being touched:

```
---
paths: ["**/*.test.tsx"]
---
# Testing conventions – only loaded when working on test files
Use React Testing Library, not Enzyme. Prefer user-event over fireEvent.

---
paths: ["src/api/**/*.ts"]
---
# API conventions – only loaded when working on API files
All endpoints must validate input with zod before touching the DB layer.
```

The exam payoff: a rule scoped with **paths: ["**/*.test.tsx"]** never loads while you're editing an API route, and vice versa — irrelevant conventions don't consume context on every operation.

2.3 Skills and context isolation with context: fork

A Skill declared with **context: fork** runs in an isolated subagent rather than the main session. This matters for any skill that produces verbose intermediate output — a dependency scan, a full codebase map, a large coverage report. That output stays contained inside the forked subagent's context and only a distilled result returns to the main

session, protecting the primary conversation's context budget.

2.4 Non-interactive / CI usage

The **-p** (print) flag runs Claude Code non-interactively: it processes the prompt, writes the result to stdout, and exits — the correct mode for CI/CD pipelines, cron jobs, and scripted automation. Without it, Claude Code launches its interactive REPL and a pipeline step will simply hang waiting for terminal input that never arrives.

```
claude -p "Summarize failing tests in this branch and suggest fixes" > report.txt
```

Exam trap: distractor options often include invented flags or hacks that sound plausible — a nonexistent environment variable, redirecting stdin from /dev/null as a workaround, or a nonexistent `--batch` flag. If you've actually run Claude Code in a pipeline, **-p** is a free point; if you haven't, memorize it explicitly.

2.5 MCP configuration scope

- **.mcp.json** — project-level MCP server configuration, version-controlled and shared across the whole team.
- **~/.claude.json** — personal/experimental MCP servers, not shared, useful for servers you're trialing before proposing them for the team config.
- Secrets in either file should use **environment-variable substitution** (e.g. `${GITHUB_TOKEN}`) rather than committed literal values — committing a real token into `.mcp.json` is a textbook exam distractor to reject.

2.6 Independent review & session isolation

A model instance that just generated a piece of code carries the context and reasoning that led to that code — when asked to review its own output in the same session, it tends to rationalize its earlier decisions rather than catch them. An independent session or subagent with no generation context, asked purely to review, catches more issues. This is the Claude Code analog of “don't grade your own homework,” and the exam tests it as a reason to deliberately spin up a fresh review session rather than continuing in the same thread.

2.7 Quick-reference command & config table

Item	Purpose
<code>-p / --print</code>	Non-interactive mode for CI/CD and scripting
<code>.claude/CLAUDE.md</code>	Shared, version-controlled project conventions
<code>~/.claude/CLAUDE.md</code>	Personal, local-only preferences
<code>.claude/rules/*.md</code> (with paths: frontmatter)	Conditionally-loaded, path-scoped conventions
<code>context: fork</code> (in a Skill)	Runs the skill in an isolated subagent to shield main context
<code>.mcp.json</code>	Shared, versioned MCP server config
<code>~/.claude.json</code>	Personal/experimental MCP server config
<code>\${ENV_VAR}</code> in MCP config	Secret injection without committing literal credentials

Prompt Engineering & Structured Output

The exam assumes baseline prompting competence and instead probes what happens at production scale: schema design, the difference between syntactic and semantic correctness, few-shot targeting, retry strategy limits, and when the Batch API is (and isn't) appropriate.

3.1 tool_use for structured output

Defining a JSON Schema and forcing the model to respond through a tool call (rather than free text you parse yourself) eliminates an entire class of failures: malformed JSON, missing braces, trailing commas, inconsistent key names. What it does **not** eliminate is **semantic** error — output that is perfectly valid JSON and still wrong. Two examples the exam likes:

- A **stated_total** field that doesn't actually match the sum of the **line_items** array — syntactically fine, arithmetically wrong.
- A required field populated with a **fabricated value** because the source document had no data for it, and the schema forced the model to supply *something*.

The exam's core lesson here: schema validation guarantees shape, not truth. You still need downstream checks (arithmetic validation, cross-field consistency checks, source-grounding checks) for correctness.

3.2 Schema design rules

- Mark a field **required** only if the information is genuinely always present in the input — a required field with no source data pressures the model to invent a plausible-looking value rather than surface the gap.
- Use **type: ["string", "null"]** (a nullable type) for fields that may legitimately be absent, so the model can express “not present” instead of guessing.
- For classification enums, add explicit **"unclear"** and **"other"** values rather than forcing every input into one of a fixed set of categories that might not fit — this is a common exam distractor: an enum with no escape-hatch value silently corrupts the data whenever a real input doesn't cleanly match any listed category.

3.3 Few-shot examples

For consistent structured output, well-chosen few-shot examples generally outperform longer descriptive instructions. The exam tests **targeting**: examples are most valuable when aimed at the specific cases the model actually gets wrong — ambiguous inputs, boundary cases, categories that overlap — rather than at the easy, already-reliable cases. Include the **rationale** behind each example's labeled answer, not just the answer itself, so the pattern the model should generalize is visible, not just memorized.

3.4 Retry-with-feedback: when it works and when it doesn't

A retry loop that feeds validation errors back to the model (“your total didn't match the line items, try again”) is effective when the correct information genuinely exists somewhere in the input and the first pass simply mis-extracted or miscalculated it. It is **not** effective when the needed information isn't in the source document at all — no amount of retrying recovers data that was never provided. The exam uses this distinction to test whether you'll build a retry pipeline for a problem retrying structurally cannot fix; the correct move in that case is a different one — flag the gap, request the missing source, or route to a human.

3.5 Batch API — when it fits and when it doesn't

The Batch API offers roughly 50% cost savings against standard pricing, with results returned within an up-to-24-hour window and **no latency SLA** — Anthropic makes no commitment about when within that window a given batch will complete.

Good fit	Poor fit
Overnight reports	Blocking pre-merge CI checks
Large-scale bulk classification/extraction	Anything a user is actively waiting on
Periodic tech-debt / analytics sweeps	Real-time chat or support interactions

Worked scenario

A manager proposes moving two workloads to the Batch API to cut cost: a blocking pre-merge code check, and a nightly technical-debt report. Correct answer: move the **nightly report** to Batch (it fits the overnight, non-blocking profile) and keep the **pre-merge check synchronous** — a developer sitting at their terminal cannot be told to wait up to 24 hours with no SLA before they can merge.

3.6 Summary: syntactic vs. semantic correctness

- **Syntactic correctness** (schemas / tool_use solve this): valid JSON, correct types, required fields present, valid enum values.
- **Semantic correctness** (schemas do NOT solve this): numbers that don't add up, fabricated values, internally inconsistent fields, claims not grounded in the source — needs explicit validation logic or a verification pass.

Tool Design & MCP Integration

Tool descriptions are the interface the model uses to decide which tool to call — this domain tests whether you can design that interface well, configure MCP correctly, and make tool failures recoverable rather than opaque.

4.1 Tool description quality

Minimal, generic tool descriptions are a leading cause of misrouted tool calls. Two tools described only as “Gets customer information” and “Gets order details” will be confused by the model whenever an input sits near the boundary between the two, because the descriptions give it nothing to discriminate on.

Worked scenario

Production logs show `get_customer` being called for order-related queries roughly 45% of the time it shouldn't be. The two tool descriptions are minimal and nearly identical. What's the first fix to try?

Not a routing classifier bolted in front of tool selection, and **not** merging the two tools into one. The first, root-cause fix is to **expand and differentiate the descriptions**: state each tool's expected input format, give 2–3 concrete example queries it should handle, call out edge cases, and explicitly state when to prefer this tool over its neighbor. Only escalate to heavier infrastructure (a classifier, merged tools, a disambiguation step) if better descriptions don't resolve it.

General description-writing checklist the exam rewards:

- State the expected input format precisely (not just “a customer identifier” — specify which ID type).
- Give concrete example queries the tool is meant to serve.
- Name edge cases and boundary conditions explicitly.
- State when to prefer this tool over a similarly-named neighbor.
- Keep the description length proportional to how much ambiguity exists between it and other available tools — near-duplicate tools need the most differentiation.

4.2 MCP server configuration

(Overlaps Domain 2 — the exam tests it from both the Claude Code workflow angle and the pure integration-design angle.) Project-scoped servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in `~/.claude.json`. Secrets always flow through environment-variable substitution (`${VAR_NAME}`), never as literal committed values.

4.3 Structured error responses from tools

What a tool returns on failure determines whether a coordinator can recover intelligently or has to give up. A tool that returns a bare string like **“Operation failed”** gives the calling agent nothing actionable. A tool that returns a structured error object gives the coordinator enough information to make a real decision.

```
// Weak – coordinator can't act on this
{ "error": "Operation failed" }

// Strong – coordinator has options
{
```

```
"errorCategory": "RATE_LIMITED",  
"isRetryable": true,  
"attemptedQuery": { "orderId": "A-4471" },  
"partialResults": null,  
"retryAfterMs": 2000  
}
```

With the structured version, the coordinator can decide to retry, fall back to an alternative tool or data source, or annotate the gap in its final output and continue — rather than failing the entire task on one opaque error. Exam distractors here are almost always generic status messages (“error”, “failed”, HTTP codes with no category or retryability signal) that look reasonable until you ask what the coordinator can actually do with them.

4.4 Tool design checklist

- Differentiate near-duplicate tools with concrete examples and explicit “prefer this when...” guidance.
- Keep each tool's scope narrow and single-purpose rather than one tool with many optional modes.
- Return structured, categorized errors (category, retryable flag, partial results) instead of bare failure strings.
- Version-control shared MCP config; keep secrets out of committed files via env-var substitution.
- Prefer fixing descriptions and error contracts over adding a routing/classification layer on top.

Context Management & Reliability

The smallest domain by weight but a frequent tie-breaker: it covers how context degrades over long sessions, how to keep long documents usable, and — critically — which escalation triggers are trustworthy.

5.1 Progressive summarization loses precision

As a long-running session gets summarized and re-summarized to control context length, precise values erode into vague language: exact figures become “approximately,” specific dates become “recently,” named entities get generalized. This is fine for narrative continuity but dangerous for anything transactional.

Fix: pull transactional facts — amounts, order IDs, exact dates, account numbers — into a **structured block** that lives outside the summarization pipeline and gets included verbatim in every prompt, regardless of how aggressively the surrounding narrative gets compressed.

5.2 The lost-in-the-middle effect

Long-context models reliably attend to the start and end of a long input more than the middle. A 75K-token aggregated research document with the single most important finding buried at token 40,000 is at real risk of being missed or under-weighted. Two structural fixes the exam expects:

- Put the **summary of key findings at the top** of a long document, not just at the end.
- Use **explicit section headings** throughout so the model has clear navigation anchors into the detail, rather than one undifferentiated wall of text.

5.3 Trimming tool output before it reaches the model

Tool outputs accumulate fast across a long agentic session. If a **lookup_order** call returns 40 fields and only 5 are relevant to the current task, sending all 40 into every subsequent prompt wastes context and adds noise the model has to filter itself. A **PostToolUse** hook that trims the raw tool result down to the relevant fields before it enters the conversation keeps the context footprint small without losing signal.

5.4 Escalation triggers — reliable vs. unreliable

This is a high-yield exam topic: knowing which escalation signals are trustworthy enough to build a production decision on.

Reliable triggers	Unreliable triggers (common distractors)
Explicit customer/user request for a human	Sentiment analysis of the conversation
A genuine policy gap the agent has no authority to interpret	The model's self-reported confidence score
Demonstrated inability to make further progress after reasonable attempts	An automatic classifier's soft label with no hard rule behind it

The reasoning behind the split: the reliable column is either an explicit, observable fact (the user asked) or a structurally verifiable condition (there is genuinely no rule to apply, or progress has genuinely stalled after N attempts). The unreliable column depends on the model's own self-assessment or a soft statistical signal — both are exactly the kind of thing that degrades silently under distribution shift and gives false confidence. Expect the exam to offer all

three unreliable options as tempting distractors in the same question.

5.5 Context management checklist

- Keep transactional facts in a structured, unsummarized block included in every prompt.
- Front-load key findings and use section headings in any long aggregated context.
- Trim tool output to relevant fields via PostToolUse hooks rather than passing raw payloads through.
- Escalate on explicit request, genuine policy gaps, or demonstrated stuck-ness — not on sentiment or self-reported confidence.

Cross-Domain Anti-Pattern Cheat Sheet

The exam's wrong answers cluster around a small number of recurring shapes. Recognizing the shape is often faster than reasoning through the specific scenario from scratch.

Anti-pattern	Domain
Parsing assistant text for completion phrases instead of checking stop_reason	Domain 1
Capping loop iterations and treating the cap as success	Domain 1
Assuming subagents inherit coordinator context automatically	Domain 1
Relying on system-prompt wording alone for a hard financial/safety policy	Domain 1
Adding a routing classifier before trying better tool descriptions	Domain 4
Merging two confused tools instead of differentiating their descriptions	Domain 4
Returning bare 'operation failed' strings instead of structured, categorized errors	Domain 4
Adding team conventions to a personal (user-level) CLAUDE.md	Domain 2
Forgetting -p / running Claude Code interactively inside CI	Domain 2
Committing literal secrets into .mcp.json instead of env-var substitution	Domain 2 & 4
Treating a valid JSON schema response as automatically correct	Domain 3
Making every schema field required regardless of source-data availability	Domain 3
Retrying extraction when the data was never in the source document	Domain 3
Routing a blocking, latency-sensitive check through the Batch API	Domain 3
Letting precise transactional values erode through repeated summarization	Domain 5
Burying key findings in the middle of a long context with no headings	Domain 5
Escalating (or not) based on sentiment analysis or self-reported confidence	Domain 5
Letting the same session that wrote the code also review the code	Domain 2

Glossary of Terms

Agentic loop — The cycle of model call → tool execution → result return → model call, continuing until stop_reason indicates completion.

stop_reason — API field indicating why the model stopped generating: tool_use, end_turn, max_tokens, or stop_sequence.

Orchestrator / coordinator — The top-level agent responsible for decomposition, delegation, error handling, and aggregation in a multi-agent system.

Subagent — A delegated agent executing a narrow subtask in isolated context, with no automatic access to the coordinator's conversation.

Context isolation — The default state in which subagents receive nothing automatically from the coordinator; all needed information must be explicitly passed.

Hub-and-spoke topology — Multi-agent pattern with a central coordinator directing independent subagents.

Hook (PreToolUse / PostToolUse) — A programmatic interception point that runs before or after a tool call, used for deterministic policy enforcement or output trimming.

CLAUDE.md — Markdown file(s) providing persistent project or personal context to Claude Code, resolved at user, project, and directory levels.

.claude/rules/ — Directory of conditionally-loaded convention files, scoped by file-path glob patterns via YAML frontmatter.

Skill — A packaged, reusable capability for Claude Code; can run with context: fork to execute in an isolated subagent.

-p / print mode — Non-interactive Claude Code flag that runs a prompt to completion and exits, required for CI/CD and scripted use.

MCP (Model Context Protocol) — Open protocol for connecting Claude to external tools and data sources via standardized servers.

.mcp.json — Project-level, version-controlled MCP server configuration file.

~/.claude.json — Personal, non-shared MCP server configuration file.

tool_use (structured output) — Forcing model output through a defined tool/JSON schema to guarantee syntactic validity.

Syntactic vs. semantic correctness — Syntactic = valid shape/types per schema; semantic = the content is actually true/consistent — schemas only guarantee the former.

Few-shot targeting — Choosing few-shot examples aimed at the model's actual failure cases (ambiguous/boundary inputs), not its easy cases.

Batch API — Asynchronous Claude API mode offering ~50% cost savings with up to a 24-hour completion window and no latency SLA.

Progressive summarization — Repeated compression of conversation history that trades precision (exact figures, dates) for brevity over time.

Lost-in-the-middle effect — Tendency for long-context models to under-attend to information placed in the middle of a long input versus the start/end.

Escalation trigger — A condition that hands control to a human; reliable triggers are explicit and structurally verifiable, unlike sentiment or self-reported confidence.

Structured tool error — An error response carrying category, retryability, and partial-result fields so a caller can decide how to recover.

12-Week Study Plan

Roughly one hour a day, organized by domain weight so the heaviest-tested material gets the most repetition. Adjust pace to your existing hands-on experience with the Claude API, Claude Code, and MCP.

Weeks	Focus	What to actually do
Weeks 1–3	Domain 1 — Agent Architecture & Orchestration	Build a small hub-and-spoke system by hand: a coordinator that decomposes a research task, delegates to Claude, and summarizes results.
Weeks 4–5	Domain 2 — Claude Code Configuration & Workflow	Write three CLAUDE.md levels in a real repo. Write path-scoped rules in .claude/rules/. Build a workflow that uses Claude Code to generate code.
Weeks 6–7	Domain 3 — Prompt Engineering & Structured Outputs	Design a JSON schema for a real extraction task; deliberately test what happens when required fields are missing or malformed.
Week 8	Domain 4 — Tool Design & MCP Integration	Write two near-duplicate tool descriptions, observe misrouting, then fix by rewriting descriptions or adjusting tool names.
Week 9	Domain 5 — Context Management & Reliability	Run a long agent session and watch summarization erode a precise value; fix it with a structured output constraint.
Week 10	Integration review	Re-read the Cross-Domain Anti-Pattern Cheat Sheet cold and explain each entry from memory. Fix any misunderstandings.
Week 11	Practice questions	Work all 40 practice questions in this guide under light time pressure (roughly 90 seconds each). Mark questions you struggle with.
Week 12	Final review & exam logistics	Redo any missed practice questions. Skim the Anthropic Academy free courses relevant to your v

Free Anthropic Academy resources worth working through

- Claude 101 — everyday-use fundamentals
- AI Fluency: Framework & Foundations
- Introduction to Agent Skills
- Building with the Claude API — function calling, tool use, streaming, SDKs, production patterns
- Claude Code in Action
- Introduction to Model Context Protocol — build MCP servers/clients from scratch
- MCP: Advanced Topics — sampling, notifications, filesystem access, production transport

40 Practice Questions with Rationales

These are original questions written to match the exam's scenario-based, root-cause-oriented style described by public candidate accounts — they are not reproductions of actual exam content, which Anthropic does not publish. Use them to test the underlying reasoning, not to memorize literal answers.

Q1. A coordinator asked to 'summarize customer feedback across all product lines' delegates three subtasks covering only the three most-mentioned products, and the final report reads as complete. Where is the fault?

- A. In the subagents, for not asking clarifying questions
- B. ✓ In the coordinator's decomposition step, which silently narrowed scope
- C. In the synthesis step, for not flagging gaps
- D. In the user's prompt, for being under-specified

Correct answer: B. The subagents executed exactly what they were assigned; the coordinator narrowed a broad scope to a convenient subset at decomposition time without flagging the narrowing. Root-cause the decomposition step first.

Q2. Which stop_reason value should trigger your orchestration loop to execute pending tool calls and continue?

- A. end_turn
- B. max_tokens
- C. ✓ tool_use
- D. stop_sequence

Correct answer: C. tool_use indicates the model wants a tool executed; execute it, return the result, and continue the loop.

Q3. A team checks task completion by scanning the assistant's final text for phrases like 'Done!' What's wrong with this approach?

- A. It's slower than checking stop_reason
- B. It's a semantic, not syntactic check
- C. ✓ It's unreliable — phrasing varies and doesn't reflect the API's actual completion signal
- D. Nothing; it's a valid approach

Correct answer: C. Text-based completion detection is an anti-pattern; stop_reason is the reliable, structural signal for whether the model is actually done.

Q4. A refund-approval policy over a fixed dollar threshold must never be violated. Which enforcement mechanism is most reliable?

- A. A firmly worded system prompt
- B. ✓ A PreToolUse hook blocking the tool call above the threshold
- C. Asking the model to double-check itself before calling the tool
- D. A note in the CLAUDE.md file

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call; prompt wording is probabilistic and can be talked around.

Q5. In a hub-and-spoke multi-agent system, what do subagents receive from the coordinator by default?

- A. The full coordinator conversation history
- B. ✓ Nothing automatically — only what's explicitly packed into their task prompt
- C. All prior subagents' outputs
- D. Only the system prompt

Correct answer: B. Subagents run in isolated context; anything they need must be explicitly included in the delegated task prompt.

Q6. Three independent research subtasks with no dependencies between them are being run one after another. What's the likely improvement?

- A. Nothing needs to change
- B. Merge them into a single subagent
- C. ✓ Run them in parallel since they're independent
- D. Add a coordinator between them

Correct answer: C. Independent subtasks with no shared dependency are strong candidates for parallel execution to reduce latency.

Q7. A coordinator commits to a fixed plan at the start of a task and executes it without checking intermediate subagent results. What capability is missing?

- A. Context isolation
- B. ✓ Dynamic planning / replanning
- C. Structured error handling
- D. Batch processing

Correct answer: B. Dynamic planning means revising the plan mid-execution based on intermediate results; a fixed, blind plan handles ambiguity poorly.

Q8. Which of the following is the LEAST reliable signal for triggering escalation to a human in an agent system?

- A. An explicit user request for a human
- B. A genuine policy gap the agent has no authority over
- C. ✓ The model's self-reported confidence score
- D. Demonstrated inability to make progress after reasonable attempts

Correct answer: C. Self-reported confidence is a soft, model-internal signal that can be miscalibrated; the other three are explicit or structurally verifiable.

Q9. A single, fully-specified ETL pipeline with one LLM classification step in the middle is being redesigned as a multi-agent system with autonomous planning. Is this justified?

- A. Yes, agents are always a strict upgrade
- B. ✓ No — fixed, fully-knowable steps don't need model-driven autonomy; a workflow is a better fit
- C. Yes, but only with three or more subagents
- D. No, because LLMs cannot do classification

Correct answer: B. Agentic architecture is justified by open-ended scope and runtime branching; a fully-known fixed sequence is better served by a simpler workflow.

Q10. A team's coding conventions are documented but new developers never seem to follow them automatically. Investigation shows the conventions live in one senior developer's own config. Where were they most likely placed?

- A. `.claude/CLAUDE.md` (project-level)
- B. ✓ `~/.claude/CLAUDE.md` (user-level)
- C. `.claude/rules/` with no path filter
- D. The system prompt

Correct answer: B. User-level CLAUDE.md is personal and not shared; team conventions belong in the version-controlled project-level file.

Q11. Which frontmatter field in `.claude/rules/` scopes a rule to only load for test files?

- A. `scope: "test"`
- B. ✓ `paths: ["**/*.test.tsx"]`
- C. `trigger: "testing"`
- D. `context: fork`

Correct answer: B. The paths field with a glob pattern conditionally loads a rule based on file path.

Q12. A Skill performs a deep, verbose dependency scan across a large codebase. What configuration keeps that verbose output from bloating the main session's context?

- A. ✓ `context: fork`
- B. A larger `max_tokens` setting
- C. Moving the skill into `CLAUDE.md`
- D. Running it with `-p`

Correct answer: A. context: fork runs the skill in an isolated subagent, containing verbose output outside the main session.

Q13. A CI pipeline step running ``claude "fix the failing test"`` hangs indefinitely. What is most likely missing?

- A. ✓ The `-p` flag
- B. A `CLAUDE.md` file
- C. An MCP server
- D. A larger context window

Correct answer: A. Without `-p`, Claude Code launches its interactive mode and waits for terminal input that a CI runner never provides.

Q14. Where should a shared, team-wide MCP server configuration live?

- A. `~/.claude.json`
- B. ✓ `.mcp.json`, version-controlled
- C. Inside `CLAUDE.md` as free text
- D. An environment variable only

Correct answer: B. .mcp.json is the project-level, shared, version-controlled location for MCP server configuration.

Q15. A developer commits a literal API token directly into `.mcp.json`. What should have been done instead?

- A. Nothing — `.mcp.json` is private by default
- B. ✓ Used environment-variable substitution like `${API_TOKEN}`
- C. Put the token in `CLAUDE.md` instead
- D. Base64-encoded the token before committing

Correct answer: B. Secrets should be injected via environment-variable substitution, never committed as literal values.

Q16. Why might the same Claude Code session that wrote a piece of code be a poor choice to review it?

- A. It has already used its context budget
- B. ✓ It tends to rationalize its own earlier decisions rather than catch them
- C. It cannot access the file system twice
- D. Session review is not a supported feature

Correct answer: B. A session carries the reasoning that produced the code, biasing it toward justifying rather than critiquing its own output; an independent session reviews more objectively.

Q17. An experimental MCP server is being trialed by one engineer before being proposed to the team. Where should its config go?

- A. .mcp.json
- B. ✓ ~/.claude.json
- C. A directory-level CLAUDE.md
- D. It should not be configured until approved

Correct answer: B. Personal/experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared project config.

Q18. An invoice-extraction agent uses tool_use with a strict JSON schema. Output is always valid JSON, but sometimes the stated_total field doesn't match the sum of line_items. What does this indicate?

- A. The schema is broken and needs to be rewritten
- B. ✓ This is a semantic error the schema cannot catch — add a validation step
- C. tool_use should be abandoned for this task
- D. The model needs a longer system prompt

Correct answer: B. Schemas guarantee syntactic validity, not semantic correctness; arithmetic consistency needs an explicit downstream check.

Q19. A schema marks a 'customer_signature_date' field as required, but roughly 20% of source documents don't contain a signature date at all. What's the likely result?

- A. The model will always leave it blank
- B. ✓ The model is pressured to fabricate a plausible date to satisfy the required field
- C. The API will reject the schema
- D. No effect — required fields are always safe

Correct answer: B. A required field with no guaranteed source data pushes the model toward inventing a value; use a nullable type instead.

Q20. A classification schema forces every input into one of five fixed categories. Real-world inputs sometimes don't fit any of them. What should be added?

- A. A sixth, more specific category
- B. ✓ An 'unclear' or 'other' value in the enum
- C. A required free-text justification field
- D. Nothing — five categories is sufficient

Correct answer: B. Enums without an escape-hatch value force bad fits into the nearest wrong category; adding 'unclear'/'other' lets the model express genuine ambiguity.

Q21. Few-shot examples for a classification task are chosen from the easiest, most obviously-correct cases. What's the likely limitation?

- A. Examples should never include a rationale
- B. ✓ They won't help much on the ambiguous/boundary cases where the model actually struggles
- C. Few-shot examples slow down inference too much to matter
- D. This is optimal targeting

Correct answer: B. Few-shot examples are most valuable when targeted at actual failure cases (ambiguous, overlapping categories), not easy cases the model already handles.

Q22. A retry-with-feedback loop is built to fix extraction errors, but a specific field is never correctly filled no matter how many retries run. Investigation shows the field's value simply isn't present anywhere in the source document. What's the correct conclusion?

- A. Increase the retry count further
- B. ✓ Retrying can't fix missing source data — flag the gap or request the missing input instead
- C. Switch to a different model
- D. The schema needs more required fields

Correct answer: B. Retry-with-feedback only helps when the correct data exists but was mis-extracted; it cannot recover data that was never in the source.

Q23. A blocking pre-merge CI check and a nightly tech-debt report are both proposed for migration to the Batch API to save cost. What's the correct call?

- A. Move both — Batch always saves money
- B. ✓ Move the nightly report only; keep the pre-merge check synchronous
- C. Move the pre-merge check only
- D. Move neither

Correct answer: B. The Batch API has no latency SLA and up to a 24-hour window, which fits an overnight report but not a check a developer is actively blocked on.

Q24. What is the primary benefit of using tool_use with a JSON schema for structured output?

- A. It guarantees the content is factually correct
- B. ✓ It eliminates syntactic errors like malformed JSON
- C. It removes the need for any validation
- D. It makes responses faster than free text

Correct answer: B. Schema-constrained tool_use guarantees valid shape/types; it does not guarantee the content itself is correct.

Q25. Roughly what cost savings does the Batch API typically offer relative to standard synchronous pricing?

- A. 10%
- B. 25%
- C. ✓ 50%
- D. 90%

Correct answer: C. The Batch API offers roughly 50% savings, in exchange for an asynchronous window of up to 24 hours with no latency SLA.

Q26. Production logs show a `get_customer` tool being called for order-related queries a large fraction of the time it shouldn't be. The two tools' descriptions are both minimal and nearly identical. What's the first fix to try?

- A. Merge the two tools into one
- B. Build a routing classifier in front of tool selection
- C. ✓ Expand and differentiate the tool descriptions with formats, examples, and edge cases
- D. Increase the model's temperature

Correct answer: C. The root cause is under-differentiated descriptions; fix that before adding classifier infrastructure on top.

Q27. What is the main risk of a tool returning only `{"error": "Operation failed"}` on failure?

- A. It uses too many tokens
- B. ✓ The calling coordinator has no basis to decide whether to retry, fall back, or escalate
- C. It violates JSON schema rules
- D. It always causes an infinite loop

Correct answer: B. A bare failure string gives no category, retryability, or partial-result information, leaving the coordinator unable to recover intelligently.

Q28. Which fields make a structured tool error most useful to a coordinator? (choose the best single answer)

- A. A stack trace only
- B. ✓ `errorCode`, `isRetryable`, and any partial results
- C. The HTTP status code only
- D. A generic error message string

Correct answer: B. Category, retryability, and partial results give the coordinator concrete grounds for a recovery decision.

Q29. Where should a project's shared MCP server list be defined so the whole team gets it automatically?

- A. `~/.claude.json`
- B. ✓ `.mcp.json`
- C. A README file
- D. An environment variable

Correct answer: B. `.mcp.json` is the version-controlled, project-scoped MCP configuration location.

Q30. A secret API key needs to be available to an MCP server without being committed to source control. What's the correct approach?

- A. Hardcode it and add the file to a private repo
- B. ✓ Use environment-variable substitution like `${API_KEY}` in the config
- C. Email it to teammates separately
- D. Store it in `CLAUDE.md`

Correct answer: B. Environment-variable substitution keeps secrets out of committed config files entirely.

Q31. Two tools are functionally similar but meant for different input types. What's the highest-leverage description addition to reduce misrouting?

- A. A longer tool name
- B. ✓ Concrete example queries and an explicit 'prefer this tool when...' statement
- C. Removing one of the tools
- D. Increasing max_tokens

Correct answer: B. Concrete examples plus explicit preference guidance give the model the discriminating signal minimal descriptions lack.

Q32. A tool occasionally returns partial results before failing outright (e.g., 3 of 5 records found). What should the error response include to make this recoverable?

- A. Nothing beyond the failure flag
- B. ✓ The partial results actually retrieved, alongside the error category
- C. A retry count only
- D. A full re-fetch of all 5 records automatically

Correct answer: B. Surfacing partial results lets the coordinator use what succeeded and only re-address the failed portion, rather than discarding everything.

Q33. Why is it generally better to fix a tool's description before adding a classifier layer for tool selection?

- A. Classifiers are never accurate
- B. ✓ It addresses the root cause directly rather than adding infrastructure on top of a flawed foundation
- C. Descriptions are required by the API
- D. Classifiers cannot be used with Claude

Correct answer: B. The exam consistently rewards root-cause fixes (better descriptions) over compensating infrastructure (a classifier) layered on top of the actual problem.

Q34. Over a long agent session, a customer's exact refund amount of \$482.17 gets progressively summarized until it appears only as 'a refund amount.' What's the correct fix?

- A. Disable summarization entirely
- B. ✓ Pull transactional facts into a structured block that's excluded from summarization and included in every prompt
- C. Ask the model to remember better
- D. Switch to a larger context window only

Correct answer: B. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed.

Q35. A 75,000-token aggregated research document has its most important finding buried around token 40,000, with no headings. What effect is this most likely to cause?

- A. Faster processing
- B. ✓ The lost-in-the-middle effect, where the finding is under-attended to
- C. No effect on model performance
- D. A schema validation error

Correct answer: B. Long-context models attend more reliably to the start and end of an input; mid-document content without structural anchors is at higher risk of being missed.

Q36. What is the recommended structural fix for the lost-in-the-middle effect in a long aggregated document?

- A. ✓ Put key findings at the top and use explicit section headings
- B. Remove all headings to shorten the document
- C. Repeat the entire document twice
- D. Convert the document to a single unbroken paragraph

Correct answer: A. Front-loading key findings and adding navigation anchors (headings) mitigates uneven attention across a long input.

Q37. A lookup_order tool returns 40 fields when only 5 are relevant to the current task. What's the best fix to reduce context bloat?

- A. Ask the model to ignore irrelevant fields
- B. ✓ Use a PostToolUse hook to trim the result to relevant fields before it enters context
- C. Call the tool twice for redundancy
- D. Increase the context window instead

Correct answer: B. Trimming at the tool-output layer (a PostToolUse hook) removes noise before it ever consumes context, rather than relying on the model to filter it.

Q38. Which of these is a RELIABLE trigger for escalating an agent interaction to a human?

- A. A sentiment classifier flags the conversation as negative
- B. The model reports low confidence in its own answer
- C. ✓ The user explicitly asks to speak with a person
- D. An automatic soft-label classifier scores the query as 'complex'

Correct answer: C. An explicit user request is a directly observable, unambiguous signal; the others are soft, model-internal, or statistical signals prone to miscalibration.

Q39. Why is a model's self-reported confidence score considered an unreliable escalation trigger?

- A. Confidence scores are not available via the API
- B. ✓ Self-assessment can be poorly calibrated and doesn't reflect actual correctness reliably
- C. It's reliable, and this is a trick answer
- D. It requires a separate model call every time

Correct answer: B. Self-reported confidence is a soft, internally-generated signal that can diverge from actual accuracy, making it a poor foundation for a hard operational decision.

Q40. An agent has made three genuine attempts to resolve a request through valid means and cannot proceed further. What should happen?

- A. Keep retrying indefinitely
- B. ✓ Escalate — demonstrated inability to progress after reasonable attempts is a reliable trigger
- C. Silently return a best-guess answer
- D. Restart the conversation from scratch with no explanation

Correct answer: B. Demonstrated, genuine inability to progress after reasonable effort is one of the reliable, structurally verifiable escalation triggers.

Exam-Day Checklist & Final Tips

Before you sit the exam

- Confirm proctoring, ID, and access requirements through your Claude Partner Network / Skilljar portal — logistics are managed by your partner organization (e.g., the consultancy or firm sponsoring your access) and can vary.
- Confirm you understand the format: 60 scenario-based multiple-choice questions across 4 sections, closed-book, no AI assistance, passing score 720/1000.
- Re-read the Cross-Domain Anti-Pattern Cheat Sheet the morning of the exam — it's the fastest possible refresh of the exam's actual decision logic.

While taking the exam

- For any “what's the root cause” question, check decomposition/description/config placement before blaming execution — the fault is very often upstream of where the symptom shows up.
- For any “what's the first fix” question, prefer the smallest, most direct structural fix (better description, better schema field, a hook) over new infrastructure (classifier, retry wrapper, shared memory layer).
- For any policy-enforcement question involving money, safety, or compliance, prefer programmatic/hook-based enforcement over prompt wording, every time.
- For any escalation-trigger question, eliminate sentiment analysis and self-reported confidence first — they are almost always the wrong answer when offered.
- Watch for Batch API questions that quietly combine a good-fit and a bad-fit workload in the same scenario — split your answer accordingly rather than picking one blanket option.

After the exam

Regardless of outcome, the material in this guide — explicit context passing over shared memory, programmatic enforcement over prompt wording, structured errors over bare failure strings, and root-cause fixes over compensating infrastructure — reflects genuinely sound production practice for Claude-based systems, independent of the certification itself.

Good luck. This guide is unofficial and independently prepared; always cross-check current exam logistics, blueprint weighting, and policies against your official Claude Partner Network materials before sitting the exam.